第0章
学习前检查与讲解规范

以飞书 Agent / RAG 课程的讲解节奏，学习真实项目源码。

检查结论：源码结构、核心命名与职责分工适合学习。默认 Python 虚拟环境仍是旧版本；本次使用已存在的 v1 环境验证学习入口。

本章包含学习准备检查、课程与源码映射、后续排版约定，以及 RAG 分块和 Agent 工具循环两个样节。正式全套课程尚未生成。

代码基准	验证口径
72dea46 + 检查时已有修改	2026-10-07，Asia/Shanghai
README、Dockerfile、P10报告及验证脚本已有修改	本轮保留这些修改；只新增学习资料
LangChain 1.4.3 / LangGraph 1.2.12	按 requirements-dev.lock 和现有 tmp/v1-dev 核对

阅读目标：分清技术概念，找到准确源码，运行小实验，并解释结果证明了什么。

目录与使用方法

1. 学习前检查：已经具备什么 3

2. 先分清三条实际调用路径 4

3. 飞书课程与项目对应 5

4. 命名规范：保留真实职责 6

5. 源码阅读顺序与章节安排 7

6. 后续教学与排版约定 8

7. RAG 样节：为什么要分块? 9

7.3 完整离线实验与结果解释 10

8. Agent 样节：谁选择工具? 11

8.3 项目工具：参数与范围分开 12

9. Context、State 与 Checkpoint 13

10. 完整 Agent 离线实验：准备 14

11. 完整 Agent 离线实验：运行与结果 15

12. 运行准备：先选对 Python 16

13. 验证记录与学习完成标准 17

14. 参考范围与后续使用 18

首次阅读：先看 1、2、5 和 12 节；准备环境后，再运行 7、10 节的两个实验。每节独立说明前置条件，示例代码中的文件名由读者自行创建。

1. 学习前检查：已经具备什么

检查项	判断与依据
职责分层	具备。api/routes 接收请求；application 组织业务；agents/tutor 装配 Agent；infrastructure 处理存储。
真实 Agent	具备。create_tutor_agent 调用 create_agent；四个业务工具通过 ToolMessage 反馈；并非仅在目录名里写 Agent。
课程命名	总体符合。TutorContext、TutorState、TutorBudgetMiddleware、create_knowledge_point_chain 等名称反映职责。
RAG 基础	具备。Document、递归切块、Embedding、Milvus Dense 召回及 PostgreSQL 复核均有真实入口。
运行准备	需选对解释器。apps/api/.venv 仍为 0.3.30；tmp/v1-dev 对应 v1 锁文件。默认环境不能直接运行 Agent 用例。
学习资料	已有源码导读和验收报告，但不是逐步教学课。设计稿存在历史阶段描述，应结合实现和报告阅读。
验证证据	本次 94 项相关离线回归全部通过；两个教学实验通过。人工质量评分和真实服务全链路不在本轮验证范围。

无需为了像课程而重排整个项目目录。当前主要准备工作是统一学习环境、建立明确的课程映射，并按输入、处理和输出拆解复杂业务。

本检查聚焦学习规范。长资料覆盖、Worker 强制中断等历史产品限制可作为案例，不能据此把项目描述为已经完成生产级质量验收。

2. 先分清三条实际调用路径



图 0-1 三条业务路径示意。Agent 的模型和工具可以形成循环。

2.1 为什么不画成一条统一 RAG 流程?

出题的 build_generation_context 按顺序拼接资料片段，并受字符预算限制；并不是先把一道用户问题转成向量，再召回出题证据。辅导工具 search_learning_materials 才会按问题进行向量检索。

错答讲解另有来源绑定的 Chain：它读取题目已经关联的证据，再生成解释。学习记录与错题直接查询 PostgreSQL，不通过 Embedding 查询。

2.2 框架提供什么，项目增加什么?

create_agent 提供图和工具执行循环。项目额外定义工具范围、必需取证、调用预算、引用验收、会话锁及已接受检查点指针。这些是阅读项目源码时要追踪的设计。

> “固定工作流” “Agent 工具循环”和“确定性判分”都可以使用模型相关组件，但三者的决策主体不同。图必须反映实际执行路径。

3. 飞书课程与项目对应

课程主题	项目入口（相对 apps/api/study_agent）	学习边界
LangChain 核心组件	llm/models.py question_generation/chains.py	统一模型接入；Prompt、LCEL、Pydantic 输出。
Agent 入门	agents/tutor/agent.py agents/tutor/tools.py	单个学习辅导 Agent，四个只读业务工具。
Runtime	agents/tutor/context.py agents/tutor/state.py	Context 提供可信范围；State 存消息与证据。
Middleware	agents/tutor/middleware.py	异步 wrap hooks；预算、参数校验、必需取证。
RAG / 文档切分	ingestion/parsers.py ingestion/chunking.py	多格式解析、页码、标题与稳定分块 ID。
Embedding / Milvus	ingestion/embeddings.py ingestion/vector_index.py	Dense、COSINE、元数据过滤；直接使用 pymilvus。
检索与引用	application/retrieval.py agents/tutor/validation.py	数据库复核候选；提交前再次确认来源。
LangGraph 持久化	infrastructure/checkpointer.py application/tutoring.py	框架图 + PostgreSQL Saver，不是自建完整 StateGraph 教程。
RAG 评估	evaluation/metrics.py 根目录 evaluation/	检索基线与结构审计；未接入 RAGAS。

课程中的 BM25、RRF、Reranker、HyDE、MCP、Multi-Agent 与 Human-in-the-loop 不自动成为本项目能力。资料/题库双索引也不是 Dense/BM25 双路召回。必要时放在明确标注的扩展阅读中。

4. 命名规范：保留真实职责

名称 / 规则	项目例子	读者应理解的职责
函数：snake_case + 动作	create_tutor_agent build_generation_context	前者装配 Agent；后者构造出题资料字符串。
类：PascalCase + 角色	TutorService MilvusDocumentIndex	业务组织与存储适配器不是同一种对象。
Context	TutorContext	本轮服务器绑定的范围与锚点，不是模型上下文窗口。
State	TutorState	图执行中的可更新数据；按字典访问。
Schema / Draft	TutorAnswerDraft SearchInput	结构约束；Draft 仍须业务验收。
工具名：业务动作	search_learning_materials get_learning_progress	模型可选择调用的四个入口。普通内部函数不都加 @tool。
中间件：用途 + Middleware	TutorBudgetMiddleware	控制执行；不等于 FastAPI 的 HTTP middleware。
快照 / 检查点	LearningSessionSnapshotStore open_tutor_checkpointer	学习缓存与 Agent 图状态持久化要分开。
兼容名 / 物理表	WorkflowRunModel 映射 agent_runs	历史表名保留；不能从表名推断有自主 Agent。

代码沿用 langchain_core.messages 和 langchain.tools 等真实导入。课程中的重导出入口可能不同，讲解时说明版本和同类职责，不为了外观一致改动工作正常的代码。

新文档使用当前规范名，旧 alias 只在迁移说明中解释。不要把 create_、build_、get_ 机械统一：名称应表达创建对象、构建数据或取得依赖的差别。

5. 源码阅读顺序与章节安排

顺序	打开的文件 / 核心问题
1. 看全貌	README.md、config.py、api/router.py：有哪些入口，哪些参数控制行为?
2. 看资料	ingestion/parsers.py → chunking.py → embeddings.py → vector_index.py：原文件如何变成可检索片段?
3. 看出题	question_generation/chains.py → schemas.py → quality.py → tasks.py：模型输出怎样被约束、过滤和保存?
4. 看工具	application/retrieval.py、learning_queries.py → agents/tutor/tools.py：工具输入由谁提供，返回哪些事实?
5. 看 Agent	context.py → state.py → middleware.py → agent.py → validation.py：模型、工具、预算与证据如何协作?
6. 看恢复	application/tutoring.py、infrastructure/tutoring.py、checkpointer.py：一轮成功怎样提交，失败为何不进入已接受历史?
7. 看学习业务	application/study.py、adaptive_selection.py、study_explanations.py：分数、选题、复习和模型解释怎样分工?

5.1 后续 PDF 建议拆成六章

第1章：项目全貌与模型调用；第2章：资料解析、分块与向量索引；第3章：受控题库生成与质量门禁；第4章：学习辅导 Agent；第5章：状态、检查点与幂等恢复；第6章：自适应练习、评估与面试复盘。

按项目依赖安排，不强行对应原课程每一章。MCP 和多 Agent 暂不独立成章；LangGraph 原理应结合 create_agent 生成的图及恢复代码讲解。

6. 后续教学与排版约定

6.1 每节沿用这样的讲解节奏

真实问题 → 名词解析 → 应用场景 → 实现思路 → 源码对应 → 代码 → 结果解释 → 易错点 → 练习。结构可以按主题调整，不把每节写成固定表单。

名词首次出现写英文、中文和一句定义，再指出本项目职责。例如 ToolMessage 是工具反馈消息，不用另一个陌生术语代替解释。每段围绕一个问题，先写输入和输出，再展开实现。

6.2 页面风格如何模仿

保留原资料的蓝色编号、深灰正文、短段落、浅灰带行号代码块、语法颜色、强调提示和概念对照表。流程图使用矢量绘制并注明是实际流程还是概念示意。目录具备页码、点击跳转和书签。

A4 竖版，左右 45 pt；中文正文 10 pt、行距 17 pt。代码约 8.2 pt，长代码拆成连续步骤；表格控制在三列左右。图表用于解释关系，不能替代必要正文。

6.3 三类代码必须标清

类型	要求
完整教学实验	有导入、输入、执行与输出；注明执行目录和解释器，必要代码不使用省略号。
源码节选	给出文件与函数、前置依赖；告诉读者不能把节选单独运行。
教学简化示例	明确简化了哪些存储、错误处理或权限约束；不冒充生产实现。

> 课程负责讲解方式，项目源码决定事实。供应商模型名、Notebook 全局变量、示例凭据和历史依赖范围不会照抄进新的教材。

7. RAG 样节：为什么要分块?

7.1 从需求开始

用户上传一份网络学习资料，可能有几十页。既要检索相关段落，又要知道内容来自哪一页，因此不能只保存一段失去来源的长字符串。

7.2 名词解析

名词	一句解释	本项目对应
Document / 文档	正文与 metadata 的组合。	解析后携带 page_start、page_end、heading_path。
Chunk / 片段	按长度和分隔符切出的较小文本。	DocumentChunk 另有 ID、哈希、页码和顺序。
Overlap / 重叠	相邻切块可能保留部分相同内容。	减少边界语义割裂；不保证每对片段都恰好重叠指定字符。
Embedding / 向量化	把文本编码为向量，便于相似检索。	embed_documents 建索引；embed_query 编码问题。

资料索引的实际准备路径



图 0-2 学习资料索引准备流程。出题和辅导随后使用各自的业务路径。

源码：ingestion/chunking.py::build_chunks。它使用 RecursiveCharacterTextSplitter，默认配置 chunk_size=800、chunk_overlap=120，按字符长度切分；token_count 是项目的粗略估计，不是精确 tokenizer 计数。

> Milvus 召回只是候选。application/retrieval.py::search 还会查询 PostgreSQL，复核知识库归属、ready 状态和资料 ID。

7.3 完整离线实验与结果解释

在 apps/api 中自行新建 ch00_chunking.py，使用第12节给出的 v1 解释器执行。下面代码直接调用项目函数，不需要模型或数据库。

完整教学实验 / 已运行同等输入

```python
from uuid import UUID
from langchain_core.documents import Document
from study_agent.ingestion.chunking import build_chunks

# 1. 准备正文和来源信息。
docs = [Document(
    page_content=("TCP使用三次握手建立连接。"
                  "三次握手确认双方收发能力。"
                  "Redis保存可重建的学习快照。"),
    metadata={"page_start": 1, "page_end": 1,
              "heading_path": ["网络学习"]},
)]
material_id = UUID("00000000-0000-0000-0000-000000000001")

# 2. 故意用小块，便于观察分隔符的影响。
chunks = build_chunks(docs, material_id=material_id,
                      chunk_size=16, chunk_overlap=4)
again = build_chunks(docs, material_id=material_id,
                     chunk_size=16, chunk_overlap=4)

# 3. 观察正文、页码和稳定 ID。
for c in chunks:
    print(c.chunk_index, repr(c.content), c.page_start)
print("stable_ids:", [c.id for c in chunks] ==
      [c.id for c in again])
```

本次结果：得到 4 个片段，页码均为 1；同一资料、顺序和正文得到相同 ID。前三块分别是 TCP 建连、收发能力和 Redis 快照的正文；最后一块仅为“。”。

这说明切分器按长度和分隔符处理文本，分块数量不等于知识点数量。小参数暴露了标点片段；当前函数只过滤空白，不过滤纯标点。生产参数与真实长文需另行观察。

8. Agent 样节：谁选择工具?

8.1 应用场景

用户问“我最近哪些知识点掌握得不好？”只检索文档无法回答。系统需要读取真实学习进度和错题；必要时再查资料解释概念。这正是把多个只读能力提供给 Agent 的场景。

8.2 认识工具调用循环



图 0-3 工具反馈会返回模型；实际可再次检索，也可能直接结束。

组件	输入 / 输出	本项目职责
Model	消息 → 回答或 tool_calls	选择查询参数和可选工具；必需读取受 middleware 门控。
Tool	业务参数 + Runtime → 结果	只读检索资料或学习事实，范围取自服务器 Context。
ToolMessage	结果 + tool_call_id	把反馈对应到模型发出的那次调用。
ToolStrategy	TutorAnswerDraft Schema	结构化最终回复，不是第五个业务查询工具。

答后追问必须先读取绑定的作答记录；错题分析必须读进度和错题。required_tool_request 会临时限制可用工具，读取标记满足后再恢复正常选择。因此“模型可以选择工具”不等于“模型可以跳过所有规则”。

8.3 项目工具：参数与范围分开

源码节选：agents/tutor/tools.py::make_tutor_tools 内部定义。前置依赖为 queries、retrieval、cache、feedback；以下不是独立脚本。

源码节选 / 为版面换行，docstring 精简

```
1  @tool
2  async def search_learning_materials(
3      query: str, runtime: ToolRuntime[TutorContext],
4      top_k: int = 5,
5  ) -> Command[Any]:
6      """检索当前知识库有效资料，只引用返回的证据。"""
7      await queries.check_scope(runtime.context.knowledge_base_id)
8      key = (" ".join(query.casefold().split()), top_k)
9      if key not in cache:
10         cache[key] = await retrieval.search(
11             runtime.context.knowledge_base_id, query, top_k
12         )
13     evidence = cache[key]
14     return feedback(
15         runtime,
16         {"code": "OK" if evidence else "NO_EVIDENCE",
17          "evidence": evidence},
18         "materials", evidence,
19     )
```

8.4 一行一行理解

第3行：runtime 是框架注入的 ToolRuntime，不让模型填写知识库 ID。第7行：程序先确认业务范围。第8-12行：当前轮相同检索可复用结果，但预算仍计调用。第14-19行：反馈同时写 ToolMessage、证据台账和读取标记。

SearchInput 在 middleware 中额外校验 query 长度、top_k≤5 和禁止额外字段。定义 @tool 只是向模型声明入口；参数校验、数据库范围和最终引用复核仍需要项目代码。

> 资料是检索数据，不是执行指令。工具只读指的是业务能力；辅导会话、消息和检查点依然会写入持久化存储。

9. Context、State 与 Checkpoint

概念	本项目对象 / 生命周期	易错点
Runtime Context	TutorContext；本轮绑定知识库、会话、轮次和答后锚点。	不是 Prompt 文本，也不是模型上下文长度。
Agent State	TutorState；messages、evidence、evidence_flags、turn_id。	State 用字典访问；Reducer 合并工具产生的数据。
业务数据库	PostgreSQL 中的题目、作答、掌握度、辅导轮次。	有数据库不等于已经接入框架 Store。
Redis Snapshot	LearningSessionSnapshot；可从业务数据库重建。	是学习进度缓存，不是 Agent 图检查点。
Graph Checkpoint	AsyncPostgresSaver；保存图消息与状态，按 thread_id 标识。	模型执行完成的 checkpoint，不自动等于业务认可的回复。

9.1 为什么需要“已接受指针”？

图执行和辅导业务提交使用不同事务。模型可能留下中间检查点，但回答未必通过引用校验。TutorService 从 last_committed_checkpoint_id 开始，重建已提交消息，重置本轮证据；验证通过后由 repository.complete 一起提交消息、轮次状态和接受指针。

恢复器只接受与轮次相符且完整的草稿，再复核资料；不完整中断标记失败。显式重试使用新消息 ID。这个机制比“加一个 checkpointer 参数”多了一层业务一致性。

> 本章实验使用 InMemorySaver，方便离线观察；生产入口使用 PostgreSQL Saver。离线结果不证明进程重启恢复，相关历史证据应阅读验收报告。

10. 完整 Agent 离线实验：准备

自行新建 apps/api/ch00_agent_loop.py。第10、11节代码连续写入同一文件。本实验复用 tests/test_tutor_agent.py 的脚本化模型与查询 fixture，实际运行项目 Agent 工厂。它不是供应商调用示例。

完整教学实验 / 第1部分，接下一页

```python
import asyncio
import time
from uuid import uuid4
from langchain_core.messages import HumanMessage, ToolMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Overwrite
from study_agent.agents.tutor.agent import create_tutor_agent
from study_agent.agents.tutor.context import TutorContext
from study_agent.agents.tutor.middleware import RunBudget
from study_agent.agents.tutor.validation import validate_draft
from tests.test_tutor_agent import (
    ScriptedTutorModel, Queries, Retrieval,
)

async def main():
    # 1. 固定模型响应和资料，观察框架工具循环。
    model = ScriptedTutorModel()
    budget = RunBudget(time.monotonic() + 10)
    graph = create_tutor_agent(
        model, Queries(), Retrieval(), budget, InMemorySaver()
    )
    config = {"configurable": {"thread_id": str(uuid4())}}
    initial = await graph.aupdate_state(
        config,
        {"messages": [], "evidence": {},
         "evidence_flags": [], "turn_id": "initial"},
        as_node="model",
    )
    ctx = TutorContext(uuid4(), uuid4(), uuid4())
```

脚本化模型首次发出 search_learning_materials 调用；收到 ToolMessage 后发出 TutorAnswerDraft 结构化回复。模型调用由 fixture 预设，可重复检查通信和状态，不模拟真实模型推理质量。

Queries 和 Retrieval 是测试 fixture，省略真实数据库和 Milvus；InMemorySaver 只在当前进程保存状态。正式服务有范围校验、持久化和更多异常处理。

11. 完整 Agent 离线实验：运行与结果

完整教学实验 / 第2部分，接上一页

```
    # 2. 从空检查点进入本轮，重置证据和标记。
    state = await graph.ainvoke(
        {"messages": Overwrite([
            HumanMessage(content="三次握手是什么？")
        ]), "evidence": Overwrite({}),
         "evidence_flags": Overwrite([]),
         "turn_id": str(ctx.turn_id)},
        initial, context=ctx,
    )
    # 3. 通过业务验收，再观察结果。
    citations = validate_draft(
        state["structured_response"], state, "materials", False
    )
    print(budget.model_calls, budget.tool_calls)
    print(state["evidence_flags"])
    print([c["evidence_id"] for c in citations])
    print(isinstance(model.calls[1][-1], ToolMessage))
    print(budget.usage())

asyncio.run(main())
```

执行目录：D:/Study/StudyAgent/apps/api。命令：

PowerShell / 现有 v1 环境

```
& '..\..\tmp\v1-dev\Scripts\python.exe' .\ch00_agent_loop.py
```

本次同等实验结果：模型调用 2 次，业务工具调用 1 次；evidence_flags 为 materials；引用为 e1；第二次模型调用前收到了 ToolMessage。usage.total_tokens 为 None、known 为 False。

这验证了实际 create_agent 图、ToolRuntime 注入、Command 更新、ToolMessage 配对和结构化回复。未知 usage 不应写成“0 Token”或“成本为零”；这里只知道未调用外部供应商。

11.1 练习与参考思路

练习一：在本轮输出前将引用 ID 改为 forged，观察 validate_draft。参考：引用不在 evidence 台账，应拒绝。可以先运行现有 test_citation_and_required_tool_validation，再修改副本。

练习二：将进度场景的必需工具读取标记去掉。参考：格式正确仍不能通过业务验收，详见 test_citation_and_required_tool_validation。

练习三：比较 InMemorySaver 和 PostgreSQL Saver。参考：前者足够观察离线机制，后者才能持久化图状态；完整恢复还依赖应用的已接受指针和业务消息。

12. 运行准备：先选对 Python

环境	本次检查
apps/api/.venv	LangChain 0.3.30、langchain-core 0.3.86；缺少 LangGraph 与 PostgreSQL Checkpointer。Agent 测试收集出现 3 项导入错误。
tmp/v1-dev	LangChain 1.4.3、langchain-core 1.6.6、LangGraph 1.2.12、PostgreSQL Checkpointer 3.1.2；94 项相关离线用例通过。

首次学习可以使用现有 tmp/v1-dev；长期建议创建独立 .venv-learning，按锁文件安装。下面是准备命令，本轮未执行安装，也未修改旧环境。

PowerShell / 长期环境准备建议

```
Set-Location D:\Study\StudyAgent\apps\api
py -3.12 -m venv .venv-learning
& '.\.venv-learning\Scripts\python.exe' -m pip install `
  -r requirements-dev.lock
& '.\.venv-learning\Scripts\python.exe' -m pip install `
  --no-deps -e .
```

12.1 用一个现有测试确认入口

PowerShell / 本轮已验证的离线文件

```
Set-Location D:\Study\StudyAgent\apps\api
& '..\..\tmp\v1-dev\Scripts\python.exe' -m pytest `
  tests/test_tutor_agent.py `
  -o 'addopts=' -q
```

编辑器解释器也应选择对应 v1 环境。Docker 路径另需数据库迁移和显式 checkpointer 初始化；启动脚本仅启动服务，不自动补齐检查点表。参见 README 的首次启用步骤。

环境不一致是学习入口问题，并不说明升级源码失效。不要复制课程旧依赖列表或依赖不明确的裸 python 命令来覆盖当前 v1 锁文件。

13. 验证记录与学习完成标准

证据类型	本次结果 / 结论范围
源码确认	阅读模型入口、Agent 工厂、四个工具、Context、State、预算、引用、检查点、业务提交及 RAG 主链路。
本次执行	7 个学习相关测试文件，94 项通过；默认旧环境收集失败。两个完整教学实验已运行同等输入。
历史记录	2026-10-01 验收报告记录真实模型、存储和浏览器结果；其后会话记录 CI 全通过，本轮不作为重跑结果。
待验证 / 未覆盖	真实供应商兼容、数据库重启恢复、浏览器链路、人工质量评分、检索大样本效果和新的覆盖率。

本次离线文件：test_tutor_agent.py、test_tutor_acceptance.py、test_tutor_budget.py、test_question_generation.py、test_ingestion.py、test_llm_models.py、test_tutor_provider_compatibility.py。

13.1 每章读完，应能留下四项记录

① 一个真实输入和可解释输出；② 能定位到文件、函数和配置的实现路径；③ 一次离线或服务实验及其证据范围；④ 一个失败案例或技术取舍，以及需要什么数据才能进一步验证。

13.2 对这个项目的学习验收问题

能否讲清出题与辅导检索的区别？为什么 query 由模型提供而 knowledge_base_id 来自 Context？为何引用合法仍不代表回答正确？为什么 Redis 快照不能代替图 checkpoint？默认环境与锁文件不一致时，如何排查导入错误？

> 学习规范看的是可理解、可定位、可实验和不混淆事实。没有 MCP、多 Agent 或完整 RAGAS 不构成当前学习目标的缺陷。

14. 参考范围与后续使用

本次提取本地目录 15 份课程 PDF 的文本，重点阅读 Agent 入门、Runtime、Middleware、RAG 架构与查询优化、评估和 Milvus 的相关段落；代表页面渲染用于对照版式。未声称把所有课程逐页完成教学验收。

材料	本章采用的内容
第2节. Agent入门实战（AI私厨）	需求分析、实现思路、分步配置/工具/模型/检查点/测试。主要对照第1、4、6-9页。
第1节. Runtime	Context、State、ToolRuntime、Command；主要对照第1-9页。
第2节. Middleware	执行钩子及控制能力；对照第1-2页及预置/自定义中间件相关段落。
第1节 RAG Agent	知识库离线/在线路径与基础组件；主要对照第1-14页。
第2节 RAG查询优化	2-Step 与 Agentic RAG；主要对照第3-10页及优化主题索引。
第3节 RAG评估 / 第4节 Milvus向量库	检索/生成评价分工；Schema、索引、Dense/Sparse 能力与项目边界。
学习PDF编写方法与复用模板.md	教学结构、三类代码、证据分类、排版与逐页验证方法。

课程根目录：D:/Study/Langchain&RAG的飞书学习文档。方法说明位于 D:/Study/chatbot-circuit-diagram/study。相关会话：项目学习--chatbot-circuit-diagram，01a0f0f3-ed6f-7163-a4cd-3c150cb3192c。

官方核对：Runtime、Tools、Persistence。接口解释以项目锁定版本和本次离线执行为准，在线文档可能包含更新能力。

后续教材沿用本章规范，先从实际模块组织第一章，再逐步展开。源码引用保持名称稳定，每章说明代码基准、前置条件、实验类型与未验证范围。